

Screensaver de sistemas solares: de secuencial a OpenMP

Paralelización con memoria compartida de un motor de entidades orientado a datos, y medición de su speedup y eficiencia.

Proyecto 1 – Informe · Integrantes: **Hugo Barillas** – carné 23556 · **Ernesto Ascencio** – carné 23009

Repositorio: github.com/BARHUGO/CC3069-Proyecto1 · C11 · raylib · OpenMP ·

2 de septiembre de 2026

1 El problema

El motor es un ECS (Entity–Component–System) orientado a datos: una entidad es un índice entero y sus atributos viven en arreglos paralelos (*struct de arrays*) dentro de una estructura `World`. Cada fotograma se ejecutan tres sistemas por entidad — **centelleo** (senoide del brillo), **órbita** (integra el ángulo y proyecta la posición) y **vida** (destruye la entidad al agotarse); en la versión secuencial se fusionan en un solo recorrido (`sys_update`). Ese recorrido es **el único trabajo que crece con N** (número de sistemas solares); el resto del fotograma es pequeño o serial. Como cada iteración `e` sólo toca la posición `e`, el bucle no tiene dependencias entre iteraciones: es paralelo por construcción.

2 Pseudocódigo

Bucle principal, ya con la bifurcación de modo (`--sequential` por defecto, `--parallel` opcional):

```

para cada fotograma:
    dt ← tiempo desde el fotograma anterior          # acotado a 50 ms
    sys_spawn_stars(mundo, dt)                        # repone estrellas de fondo
    sys_drift(sistemas, dt)                           # mueve el centro de cada sistema
    si modo == paralelo:
        #pragma omp parallel for schedule(static, 64) num_threads(T)
        para e en 0 .. entidades-1:
            centelleo(e); órbita(e)
            si vida(e) llegó a 0: marcar e con bit C_PENDING_DESTROY
        # ← barrera implícita: todos los hilos terminan aquí
    si no:
        para e en 0 .. entidades-1: centelleo(e); órbita(e); vida(e)
    destruir entidades marcadas                        # en serie, hilo principal
    sys_trails(mundo, dt)                             # muestrea posiciones para estelas
    sys_render(mundo, sistemas, estelas)             # sólo hilo principal (OpenGL)
    dibujar FPS en pantalla

```

Sincronía. Toda la paralelización cabe en la única directiva `#pragma omp parallel for schedule(static, 64)` mostrada arriba, y su único mecanismo de sincronía es la barrera implícita al cerrar la región. No se usan mutex, secciones críticas ni atómicas. La destrucción de entidades — la única escritura a estado compartido (`freeList`, `alive`) — se difiere con *marcar-y-barrer*: dentro de la región cada hilo sólo activa el bit `C_PENDING_DESTROY` en su propia posición `mask[e]`, y un segundo recorrido serial, fuera de la región, hace las bajas reales. La ruta paralela cae a la secuencial por debajo de **6144 entidades**, donde abrir la región cuesta más que el trabajo.

3 Método PCAM

Tabla 1. PCAM aplicado al bucle de actualización.

FASE	DECISIÓN
Particionar	La unidad de trabajo es una entidad. El recorrido por entidad que fusiona centelleo + órbita + vida es lo único que crece con <code>N</code> ; es el dominio a repartir.
Comunicar	Ninguna comunicación entre tareas: con el layout SoA la iteración <code>e</code> sólo toca la posición <code>e</code> . Las altas y bajas de entidades se sacan de la región (las bajas se marcan con un bit y se aplican después).
Aglomerar	Las tres operaciones por entidad se agrupan en un único bucle paralelo con reparto <code>static</code> en bloques contiguos de 64 iteraciones: cada hilo procesa un tramo del arreglo, lo que conserva la localidad de caché.
Mapear	Por defecto 4 hilos; <code>--threads T</code> fuerza otro número (incluso más que los núcleos, para medir el límite). Una barrera implícita al cerrar la región; tras ella el hilo principal aplica bajas, muestrea estelas y dibuja.

4 Metodología de medición

El binario incorpora un modo `--benchmark` : descarta 3 s de calentamiento y mide durante 10 s tomando una muestra por segundo. La métrica para el speedup es el **tiempo mediano de actualización ECS por fotograma, aislado del render** — no los FPS totales, porque el render es serial y domina el fotograma. Un script (`scripts/benchmark.sh`) corre el binario **10 veces** por configuración y agrega media, mediana y desviación estándar; en modo paralelo barre los conteos de hilos 1, 2, 4, 8, 16. Escena fija en todas las corridas: semilla `20260831` , 1280×720, 500 estrellas de fondo, `--no-vsync` . Speedup $S = t_{seq} / t_{par}$; eficiencia $E = S / hilos$.
Máquina: **Apple M5 Pro, 15 núcleos**, macOS 26.6.2, Homebrew clang 22. Cada valor reportado es la mediana de 10 corridas.

5 Resultados

Tabla 2. Tiempo de actualización ECS a 4 hilos (la «rodilla» de la curva).

N	T_SEQ (MS)	T_PAR 4H (MS)	SPEEDUP	EFICIENCIA	FPS SEQ	FPS PAR
1 000	0.086	0.108	0.80×	20 %	55.3	36.6
5 000	0.461	0.223	2.07×	52 %	12.6	12.6
15 000	1.392	0.648	2.15×	54 %	5.5	5.5

Lectura. A $N = 1000$ (~6500 entidades, apenas sobre el umbral) las 4 hilos quedan por debajo del secuencial: arrancar la región cuesta casi tanto como el trabajo. Los FPS totales no cambian entre modos porque los fija el render. A $N = 1\,000\,000$ (~6 millones de entidades, sin `--benchmark`) la actualización toma ~104 ms; el programa no se rompe a esa escala (~0.7 GB).

Tabla 3. Speedup por N y número de hilos.

N	1 HILO	2 HILOS	4 HILOS	8 HILOS	16 HILOS
1 000	0.32×	0.26×	0.80×	1.31×	1.15×
5 000	1.01×	1.61×	2.07×	1.52×	1.52×
15 000	1.03×	1.64×	2.15×	2.29×	2.19×

Lectura. El speedup crece hasta 4 hilos y ahí se aplana; el pico (2.29× a 15 000) está en 8, y a 16 hilos empeora. La marca resalta el mejor valor de cada fila.

Tabla 4. Eficiencia (speedup ÷ hilos).

N	1 HILO	2 HILOS	4 HILOS	8 HILOS	16 HILOS
1 000	32 %	13 %	20 %	16 %	7 %
5 000	101 %	81 %	52 %	19 %	10 %
15 000	103 %	82 %	54 %	29 %	14 %

Lectura. La eficiencia baja de forma monótona con cada hilo extra. El >100 % a 1 hilo para N grande es un efecto de caché: la región paralela con un solo hilo ya reordena el acceso mejor que el bucle secuencial.

6 Análisis y conclusiones

- **La actualización escala hasta un punto:** $2.15 \times N = 15\,000$ con 4 hilos, sin ningún mecanismo de exclusión mutua. El patrón por entidad sobre SoA era paralelizable de raíz.
- **El límite no es el número de núcleos, es la memoria:** pasar de 4 a 8 o 16 hilos casi no mejora el tiempo y la eficiencia se desploma (54 % $\rightarrow$ 29 % $\rightarrow$ 14 %). El bucle mueve más bytes de los que calcula y todos los hilos comparten el mismo ancho de banda.
- **Los FPS totales no se mueven:** como predice la ley de Amdahl, la fracción serial — el render, que raylib y OpenGL exigen en un solo hilo — fija el techo. En el otro extremo, a $N = 1000$ el overhead de crear el equipo de hilos domina y el speedup es < 1 : de ahí el umbral de 6144 entidades. El diseño orientado a datos fue lo que permitió paralelizar con una sola directiva y sin exclusión mutua, con equivalencia bit a bit verificada en las pruebas.

7 Pruebas

Tabla 5. Pruebas unitarias (`make test`) — todas pasan.

CASO	QUÉ VERIFICA	RESULTADO
Equiv. ruta corta	<code>sys_update</code> vs <code>sys_update_parallel</code> con mundo pequeño (cae a la ruta secuencial): <code>World</code> completo idéntico tras un paso.	OK
Equiv. ruta OpenMP	> 6144 entidades con 8 hilos (sobresuscripción): <code>World</code> bit a bit igual al de 1 hilo.	OK
Estelas y RNG	<code>TrailBuffer</code> y estado del <code>Rng</code> idénticos en ambas rutas tras el paso.	OK
Conteo de bajas	Número de entidades destruidas igual en secuencial y paralelo, y coincide con las estrellas objetivo.	OK
Límites de layout	<code>assert_layout_bounds</code> : posiciones y radios dentro de rango tras spawn.	OK
Regresión de deriva	<code>assert_drift_visible</code> : al menos un sistema se desplaza > 20 px en 2 s de simulación.	OK
Profundidad estable	<code>assert_depth_survives_churn</code> : la capa de profundidad se conserva tras altas/bajas repetidas.	OK
Deathstar	Movimiento WASD acotado a pantalla; un solo disparo por ciclo; el impacto destruye el sistema objetivo.	OK

Tabla 6. Bitácora de mediciones — resumen (mediana de 10 corridas, ms de actualización ECS por fotograma).

N	HILOS	T_SEQ	T_PAR	Σ PAR	SPEEDUP	EFICIENCIA
15 000	4	1.390	0.645	0.025	2.15×	54 %
15 000	8	1.390	0.595	0.040	2.29×	29 %
5 000	4	0.461	0.223	0.006	2.07×	52 %
1 000	8	0.085	0.063	0.006	1.31×	16 %

Nota. A $N = 1000$ el ruido es mayor y a 1, 2 y 4 hilos el paralelismo queda por debajo del secuencial (Tabla 3). Los CSV completos, una fila por corrida, están versionados en [benchmark-results/](#).

8 Referencias

1. OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. 2021. openmp.org/spec-html/5.2
2. Foster, I. *Designing and Building Parallel Programs*. Addison-Wesley, 1995. Cap. 2, «Designing Parallel Algorithms» (metodología PCAM). mcs.anl.gov/~itf/dbpp
3. Amdahl, G. M. «Validity of the single processor approach to achieving large scale computing capabilities». *AFIPS Conference Proceedings*, vol. 30, 1967, pp. 483–485.

Anexo — Diagrama de flujo

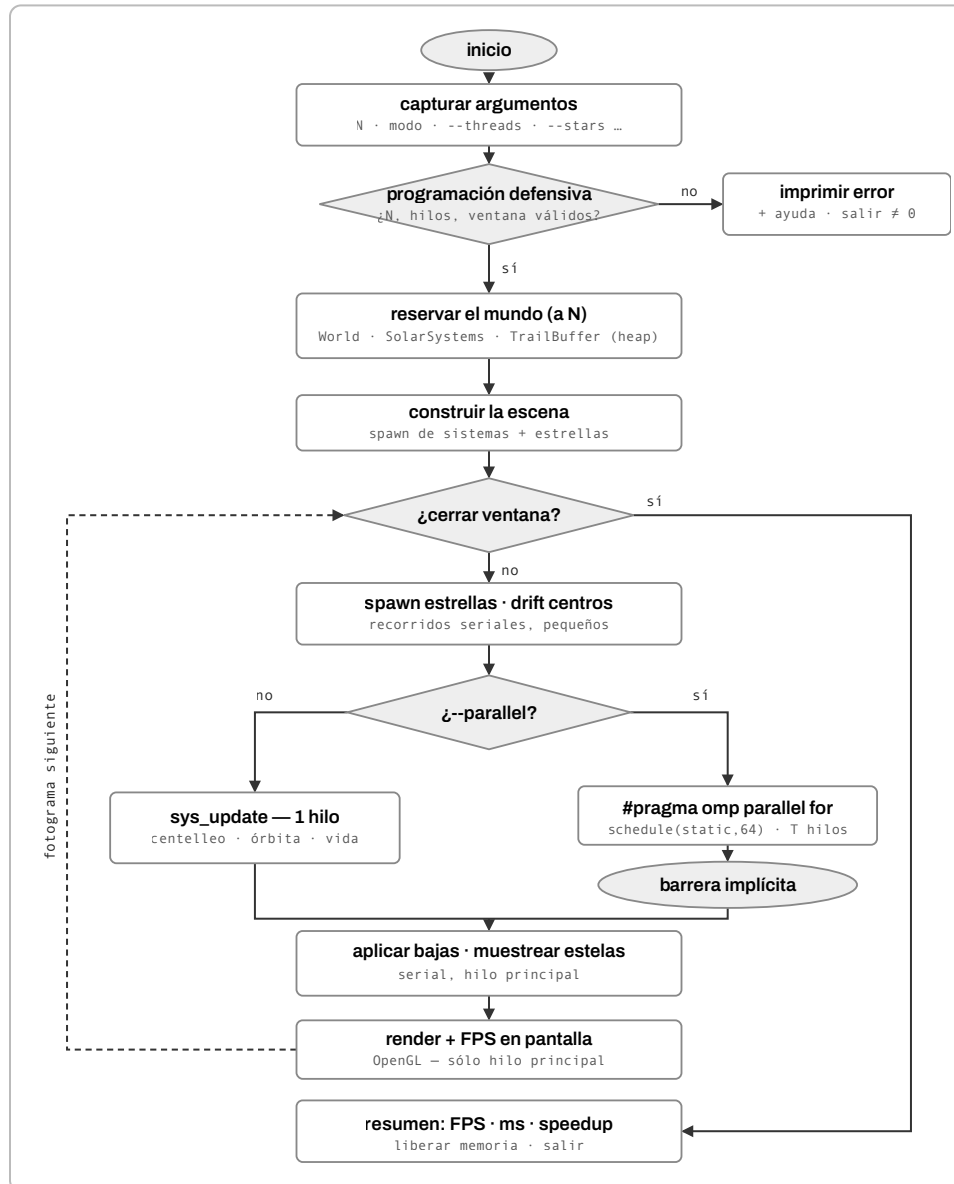


Figura 1. Diagrama de flujo. El rombo «¿--parallel?» separa las dos rutas de actualización; la única sincronía es la barrera implícita al cerrar la región. La destrucción de entidades y el render quedan siempre en el hilo principal. La ruta paralela sólo se toma con más de 6144 entidades.